

A TECHNICAL WHITEPAPER

Structural Intelligence Layer

Making blockchain structure queryable

PUBLISHED
BY

Simplex
TDA

VERSION

v4

SUBJECT

Topological data analysis for
blockchain networks

CONTACT

simplex_tda@protonmail.com

Contents

1. Introduction: the missing structural layer
2. Position in the existing data stack
3. From blockchain activity to structural representation
4. Structural primitives
5. Empirical validation
6. Production feasibility and compute economics
7. SIL architecture
8. Primitive discovery, calibration, and validation
9. Machine-readable structural context
10. Applications and structural state delivery
11. Open methodology and interoperability
12. Limitations and open questions
13. Initial deployment roadmap
14. Long-term vision

Conclusion · Appendix A: mathematical terms and primitive glossary

Abstract

Blockchain data infrastructure has become highly effective at making individual components of blockchain activity accessible. Developers can query transactions, events, balances, contracts, wallets, protocol activity, and labeled entities through nodes, indexers, analytics platforms, and application-specific data services.

However, an important class of information remains difficult to access directly: the **structural state of the network itself**.

A blockchain is not merely a collection of independent transactions. Transactions form evolving networks of relationships between addresses, contracts, protocols, liquidity pools, and other entities. These networks may become more fragmented, concentrated, interconnected, or structurally unstable even when conventional aggregate metrics remain relatively unchanged.

The **Structural Intelligence Layer (SIL)** is a proposed open infrastructure layer that transforms blockchain activity into reusable, machine-readable descriptors of network structure.

SIL combines graph-based representations with Topological Data Analysis (TDA) and related structural methods to produce versioned primitives describing properties such as:

- fragmentation;
- connectivity;
- persistence;
- complexity;
- structural drift;
- stability;
- anomalous structural change.

The objective is not to replace blockchain indexers, graph databases, or analytics platforms. Rather, SIL is designed as a complementary layer that consumes structured blockchain data and publishes **precomputed structural state** for downstream applications.

The methodology is motivated by empirical work conducted on Ethereum across three distinct tasks: structural anomaly detection, volatility prediction, and MEV classification. Across different datasets, time scales, and mathematical representations, these experiments found that topological and structural features contained measurable information relevant to the task under evaluation. In the MEV experiment, TDA features achieved a ROC-AUC of 0.942 alone and 0.963 when combined with conventional transaction features.

The initial production model is based on **daily aggregation and offline computation**. Persistent homology is not performed at query time. Each daily structural snapshot is computed, calibrated, validated, and stored before being exposed through APIs and SDKs. Consequently, computational cost and user-facing query latency are treated as separate engineering problems.

The first deployment will explicitly benchmark the computational economics of candidate representations and publish measurements of input size, runtime, memory consumption, and output stability.

The central proposition of SIL is therefore:

Blockchain infrastructure should make not only blockchain activity queryable, but also aspects of blockchain structure.

1. Introduction: The Missing Structural Layer

Blockchain ecosystems generate enormous quantities of machine-readable data.

Existing infrastructure allows developers to ask questions such as:

- Which transactions occurred?
- Which contracts were called?
- How much value moved?
- Which wallets interacted?
- What events were emitted?
- What is the current liquidity or protocol state?

These questions describe the **components** of blockchain activity.

They do not necessarily provide a compact description of how those components are collectively organizing into a changing network.

Consider two periods with similar:

- transaction counts;
- gas usage;
- trading volume;
- total value locked;
- throughput.

The underlying interaction network may nevertheless have changed substantially.

Activity may have become concentrated around fewer entities. Previously disconnected regions may have become connected. A new protocol may have created new interaction pathways. A migration may have reorganized liquidity across the ecosystem.

These changes can be difficult to describe using any single conventional metric.

Answering a structural question such as:

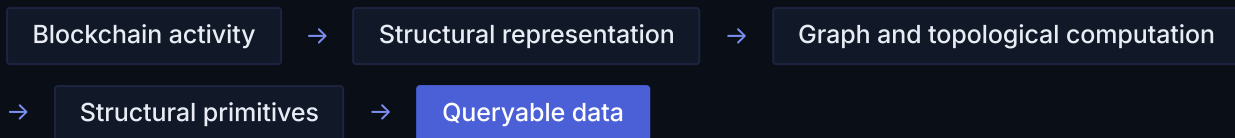
How different is today's transaction network from its recent historical state?

typically requires an application developer to independently:

1. acquire blockchain data;
2. define a structural representation;
3. construct the relevant graph;
4. perform structural computation;
5. maintain the computational pipeline;
6. store historical results;
7. define a methodology for comparing structural states.

The Structural Intelligence Layer proposes to standardize and centralize this transformation.

Conceptually:



The objective is to transform an expensive and specialized analytical process into reusable infrastructure.

2. Position in the Existing Data Stack

SIL is not intended to replace existing blockchain data infrastructure.

Its proposed role is a layer **above raw blockchain data and indexing**, producing a derived dataset that applications can consume.

INFRASTRUCTURE LAYER	PRIMARY FUNCTION	TYPICAL OUTPUT
Blockchain nodes	Access chain state	Blocks, transactions, logs
Indexing infrastructure	Organize blockchain data	Queryable entities and events
Analytics platforms	Aggregate and analyze activity	Metrics, dashboards, SQL results
Graph infrastructure	Represent relationships	Nodes, edges, communities, paths
Structural Intelligence Layer	Compute and publish structural state	Fragmentation, persistence, drift, stability, anomaly signals

The Graph, for example, provides infrastructure for indexing, organizing, streaming, and querying blockchain data. These capabilities can provide inputs to higher-level structural computation; they do not by themselves define a standardized structural primitive layer.

Similarly, graph analytics systems provide methods for community detection, pathfinding, similarity, centrality, and other graph operations.

SIL is complementary to both.

The distinction is:

Existing infrastructure makes underlying data and relationships available for analysis. SIL proposes to repeatedly compute selected higher-order structural descriptors and expose the resulting state directly.

A developer should not need to implement a persistent-homology pipeline simply to ask:

Has the structural state of the network changed significantly?

Instead, the developer could query a precomputed structural primitive.

2.1 Relationship to Blockchain Indexers

Indexers solve the problem of transforming raw blockchain state into queryable data.

SIL depends on this capability.

A SIL deployment may consume data from:

- chain-native indexing systems;
- blockchain data providers;
- subgraphs;
- streaming infrastructure;
- RPC providers;
- archive nodes.

SIL does not attempt to compete with these systems at the ingestion layer.

Its role begins after relevant blockchain activity has been made available.

2.2 Relationship to Analytics Platforms

Analytics platforms aggregate blockchain data into metrics, dashboards, SQL queries, and application-specific insights.

SIL proposes a different kind of derived output: **structural state variables**.

These variables can themselves become inputs to analytics platforms.

For example, an analytics application could combine:

- volume;
- TVL;
- active addresses;
- transaction count;

with:

- structural fragmentation;
- persistence entropy;
- structural drift;
- anomaly scores.

SIL therefore extends rather than replaces existing analytics.

2.3 Relationship to Graph Analytics

Graph analytics already provides a mature family of methods for analyzing:

- centrality;
- connectivity;
- shortest paths;
- communities;
- similarity;
- local neighborhoods.

SIL does not claim that TDA replaces graph analytics.

Instead, it proposes a structural layer in which graph-derived and topology-derived descriptors can coexist.

Potential primitive families include:

GRAPH-DERIVED DESCRIPTORS

- component structure;
- connectivity;
- concentration;
- clustering.

TOPOLOGY-DERIVED DESCRIPTORS

- Betti numbers;
- persistence entropy;
- total persistence;
- lifetime statistics;
- persistence-diagram distances.

TEMPORAL DESCRIPTORS

- structural drift;
- topology velocity;
- stability;
- anomaly scores.

The central idea is not:

TDA instead of graph analytics.

It is:

A reusable structural feature layer built from graph methods, topology, and temporal analysis.

3. From Blockchain Activity to Structural Representation

A blockchain can be represented mathematically in multiple ways.

There is no single canonical representation.

For the initial SIL architecture, the primary representations are graph and network-based, with the specific construction selected according to the structural question being investigated.

Possible representations include:

ADDRESS INTERACTION GRAPHS

Nodes represent addresses and edges represent observed interactions.

WEIGHTED INTERACTION GRAPHS

Edges may encode:

- transaction count;
- transferred value;
- interaction frequency;
- temporal information.

CONTRACT INTERACTION GRAPHS

Nodes represent externally owned accounts or contracts, while edges represent calls or other interactions.

PROTOCOL-LEVEL GRAPHS

Activity is aggregated around:

- protocols;
- liquidity pools;
- applications;
- other higher-level entities.

MULTILAYER NETWORKS

Different transaction types or categories form separate but related network layers.

The choice of representation determines what structural information is available.

For this reason, SIL is not based on the assumption that one graph construction is universally optimal.

Instead:

Representation selection is itself an empirical component of the methodology.

Every published primitive should therefore be associated with a documented and versioned representation.

Candidate representations are evaluated according to:

- computational cost;
- stability;

- interpretability;
- sensitivity to structural change;
- incremental information beyond conventional features;
- relevance to downstream applications.

Other mathematical representations may be evaluated for specific research or downstream applications, but are not assumed to form part of the initial SIL production architecture.

4. Structural Primitives

A structural primitive is a machine-readable descriptor representing a specific property of a blockchain interaction system.

Primitives are not computed in a vacuum on a single monolithic dataset; they are calculated per mathematical **layer**—where a layer represents a distinct graph construction or filtration scale (e.g., Layer 0: Address-to-Address interaction, Layer 1: Protocol-level value flows). The target layers and initial primitive vocabulary established at launch are intentionally modular and not static. Instead, both primitives and underlying layer definitions evolve continuously through an iterative feedback loop of empirical evaluation and direct correspondence with downstream ecosystem applications.

Potential primitive families include:

PRIMITIVE	POSSIBLE MATHEMATICAL BASIS	STRUCTURAL INTERPRETATION
Complexity	Total persistence	Total volume of persistent structural features
Stability	Mean or aggregate feature lifetime	Persistence of observed interaction patterns
Fragmentation	Zeroth Betti number or component structure	Degree of structural separation or isolation
Cyclicity	First Betti number or related graph measures	Presence of loops or recurring interaction pathways
State Diversity	Persistence entropy	Distribution of structural features across scales
Structural Drift	Wasserstein or bottleneck distance	Change relative to a previous or baseline structural state
Anomaly Score	Statistical analysis of structural time series	Degree of unusual structural deviation

These examples are not intended to define a universal or fixed primitive set.

Primitive selection and representation selection are empirical components of the methodology.

A candidate primitive must therefore be evaluated not only for mathematical validity, but also for:

- interpretability;
- stability;
- reproducibility;
- computational feasibility;
- measurable information value.

Every published primitive should include metadata describing:

- the network;
- the aggregation period;

- the underlying representation;
- methodology and parameter versions;
- relevant data coverage.

This allows primitives to function as reproducible data products rather than unexplained numerical outputs.

5. Empirical Validation

The Structural Intelligence Layer is based on a specific empirical hypothesis:

Can structural and topological representations of blockchain activity contain useful information that is not fully captured by selected conventional features?

Simplex TDA has investigated this question across three distinct Ethereum experiments.

These experiments do not validate the complete SIL infrastructure. They test the underlying methodological proposition across different:

- datasets;
- time scales;
- mathematical representations;
- downstream tasks.

EXPERIMENT	REPRESENTATION	TASK	MAIN RESULT
Structural anomaly detection	Daily transaction-network topology	Detect structural transitions	Major historical disruptions reflected in structural signals
Predictive validation	Daily persistence-diagram features	Future volatility prediction	Statistically significant incremental information in evaluated settings
MEV detection	Per-block transaction point clouds	Classification	0.942 ROC-AUC with TDA; 0.963 combined

The experiments do not represent repeated measurements of a single benchmark.

They examine whether structural information remains useful under substantially different conditions.

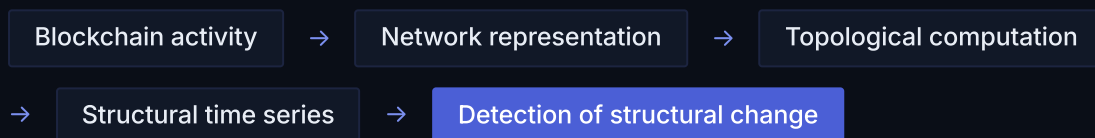
5.1 Structural Anomaly Detection

The initial prototype applied persistent-homology-based analysis to evolving Ethereum transaction networks.

The objective was to identify periods in which network structure differed substantially from historical behavior.

The prototype identified major historical structural transitions, including Black Thursday and The Merge.

The importance of this experiment for SIL is architectural:



The result should not be interpreted as a universal early-warning system.

It demonstrates that significant changes in blockchain activity can be reflected in measurable changes in network structure.

5.2 Predictive Validation

A subsequent experiment tested whether topological features derived from Ethereum transaction networks contained information relevant to a future prediction task.

Daily persistence-diagram features were evaluated using temporally appropriate validation, placebo controls, robustness checks, and statistical testing.

The experiment found statistically significant evidence that multiple structural feature families contained information relevant to future volatility beyond the conventional baselines evaluated.

The claim is deliberately limited:

In the evaluated setting, temporal structural features contained information relevant to future volatility that was not fully captured by the conventional feature sets tested.

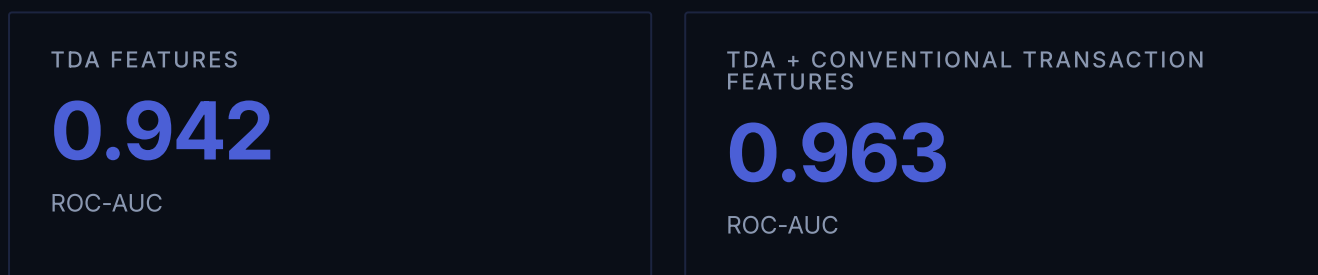
This suggests that structural primitives may function not only as descriptive indicators, but also as inputs to downstream systems.

5.3 Representation Transfer: MEV Detection

The third experiment used a fundamentally different representation.

Transactions within individual Ethereum blocks were represented as transaction-level point clouds, from which topological features were computed for MEV-related classification.

This representation is experimental and is not part of the initial SIL production architecture. Its purpose is to test whether structural information can remain useful when blockchain activity is represented geometrically rather than as an interaction graph.



The incremental contribution of topological features beyond conventional transaction features was statistically significant in the evaluated setting.

This is important because it demonstrates that the broader structural methodology is not necessarily limited to a single mathematical representation.

Together, the three experiments span:

- network-level structural change;
- predictive modeling;
- classification;

using different representations of blockchain activity.

This supports a central methodological principle:

Blockchain representation should remain an experimentally evaluated design choice rather than a fixed assumption.

The MEV point-cloud experiment should therefore be understood as validation of representation transfer, rather than as a specification of the SIL production data model.

5.4 What Has—and Has Not—Been Validated

The existing experiments provide evidence that structural representations can contain useful information.

They do not yet establish SIL as a production infrastructure layer.

Open questions include:

- Which representations provide the best information-to-compute-cost trade-off?
- Which primitives are sufficiently stable and interpretable for production use?
- Which methods generalize across networks?
- Which downstream applications derive measurable utility?

The purpose of SIL is to move from research validation toward answering these questions systematically.

6. Production Feasibility and Compute Economics

Persistent homology can be computationally expensive.

SIL therefore does not propose to perform topology computation synchronously in response to user queries.

The initial production model uses **daily aggregation and offline computation**.

This separates:

1. **structural computation latency**; and
2. **user-facing query latency**.

6.1 Daily Batch Architecture

The initial production cycle is:

- 01 Day N blockchain activity
- 02 Data extraction
- 03 Representation construction
- 04 Graph and structural computation
- 05 Primitive generation
- 06 Calibration and validation
- 07 Versioned storage
- 08 API availability

Once a structural snapshot has been computed, downstream applications retrieve the stored result.

A user query does not trigger persistent-homology computation.

The cost of producing structural state and the latency of querying it are therefore separate engineering problems.

6.2 Append-Only Computation

The initial system does not require fully incremental persistent homology.

For non-overlapping daily windows:

```
Day 1 → compute → store  
Day 2 → compute → store  
Day 3 → compute → store
```

A new observation does not require recomputing the complete historical dataset.

The system is therefore append-only at the dataset level, even if each daily representation is initially computed independently.

Future versions may investigate:

- incremental computation;
- rolling-window updates;
- streaming methods;
- higher-frequency structural state.

These are optimizations rather than prerequisites for the initial deployment.

6.3 Representation-Level Cost Control

The primary approach to computational scaling is to manage complexity at the representation layer.

Potential strategies include:

- daily rather than real-time aggregation;
- low-dimensional topology;
- sparse representations;
- reproducible sampling or coarsening;
- approximation where empirically justified;
- representation-specific computational pipelines.

Not every representation requires the same structural method or computational budget.

6.4 Structural Compute Benchmarking

SIL does not assume a universal production cost before chain-specific benchmarking.

The first deployment will therefore publish a **Structural Compute Benchmark Report** measuring candidate representations.

METRIC	MEASUREMENT
Input size	Nodes, edges, or observations
Density	Relevant sparsity characteristics
Runtime	Wall-clock computation time
Memory	Peak memory consumption
Hardware	CPU/GPU and memory configuration
Primitive stability	Sensitivity to sampling or coarsening
Output size	Published dataset size

The initial production representation will be selected according to an empirically measured trade-off between:

Information value × computational cost × reproducibility

7. SIL Architecture

The proposed architecture consists of five layers.

7.1 Data Layer

Blockchain activity is obtained from appropriate chain-native or external data sources.

The architecture is intended to remain upstream-provider agnostic.

7.2 Representation Layer

Raw activity is transformed into a documented and versioned graph or network representation.

Examples include:

- address interaction graphs;

- contract interaction graphs;
- protocol graphs;
- weighted interaction networks;
- multilayer interaction networks.

The specific representation is selected through the empirical discovery and validation process described in Section 8.

7.3 Structural Compute Layer

Selected graph, topological, and temporal computations are performed.

Potential components include:

- graph connectivity;
- component structure;
- persistent homology;
- persistence summaries;
- distance metrics;
- temporal anomaly detection.

7.4 Primitive Store

Results are stored as versioned time-series observations.

Each observation includes relevant metadata describing:

- network;
- aggregation period;
- representation;
- methodology version;
- parameters;
- data coverage.

7.5 Access Layer

Applications access precomputed structural state through:

- REST;
- GraphQL;
- Python SDKs;
- TypeScript SDKs.

The central architectural principle is:

The access layer retrieves precomputed structural state. It does not perform topology computation during a user request.

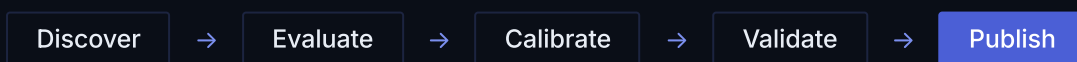
8. Primitive Discovery, Calibration, and Validation

Structural primitives are not assumed to have universal meanings across blockchain networks.

A persistence entropy of `0.61`, a structural drift of `0.12`, or a fragmentation score of `0.31` has limited meaning without reference to the network, representation, aggregation period, historical distribution, and methodology that produced it.

For this reason, deploying SIL to a new blockchain does not begin by computing a fixed predefined metric set.

It begins with an empirical process:



The objective is to determine:

1. Which structural representations are meaningful for the network?
2. Which primitives are stable, interpretable, reproducible, and computationally practical?
3. Which primitives contain information beyond simpler measures of network activity?
4. Which primitives demonstrate measurable value for a defined downstream use case?

Only candidates that perform sufficiently well under this process are promoted to the production dataset.

8.1 Discovery and Representation Selection

A network may be examined through multiple structural layers, including:

- value-transfer networks;
- contract interactions;
- protocol-level graphs;
- transaction categories;
- address interaction networks;
- weighted or multilayer interaction networks.

Not every representation will be equally useful.

Candidate representations are evaluated according to:

- information value;
- stability;
- interpretability;
- reproducibility;
- computational feasibility.

Candidate primitives are also tested against simpler explanations.

A structural feature may appear informative merely because it tracks transaction volume, network size, or general activity. Where appropriate, SIL evaluates whether structural features retain useful information after controlling for relevant confounding variables and comparing them with conventional metrics.

The objective is not to identify the most mathematically complex representation, but the structural measurements that provide useful information otherwise unavailable.

8.2 Network-Specific Calibration

Raw values should generally be interpreted relative to network-specific reference distributions.

Depending on the primitive, calibration may include:

- percentiles and quantiles;
- fixed or rolling historical baselines;
- standardized or robust deviation scores;
- temporal trends and regime changes.

For example:

```
{
  "persistence_entropy": 0.61,
  "historical_percentile": 94,
  "rolling_deviation": 2.3
}
```

This allows an application to interpret a measurement not simply as:

```
structural_drift = 0.12
```

but as:

Structural drift is unusually high relative to the network's recent historical behavior.

SIL does not assume universal thresholds for "normal" or "anomalous" behavior. Calibration is network-, representation-, and application-specific.

8.3 Historical Validation

Candidate primitives can be evaluated against independently identified historical events, including:

- upgrades;

- migrations;
- exploits;
- liquidity events;
- market dislocations;
- ecosystem transitions.

The purpose is not to tune metrics until they reproduce known events.

Instead, historical events provide an external reference:

Do independently identified changes in network behavior correspond to measurable changes in structural state?

Both positive and negative results are informative.

A lack of response may indicate that the selected representation is insensitive to that type of change.

Where possible, event validation should use predefined event sets and transparent reporting to reduce retrospective overfitting.

8.4 Downstream and Incremental Validation

Historical correspondence alone does not establish practical value.

Candidate primitives should therefore also be evaluated against concrete downstream tasks.

MODEL	INPUTS
Baseline	Conventional metrics
Structural	Structural primitives
Combined	Conventional + structural metrics

The central question is not:

Can topology produce a signal?

It is:

Does structural information provide incremental value beyond conventional data?

Depending on the application, validation may involve:

- out-of-sample anomaly detection;
- walk-forward prediction;
- classification;
- simulations or backtesting of protocol parameter logic;
- ablation and robustness analysis.

Only primitives that demonstrate sufficient stability and measurable information value should be promoted from experimental analysis to the production dataset.

8.5 Baseline Evolution and Versioning

Blockchain ecosystems evolve.

A historical baseline that accurately describes a network at one stage of its development may become less relevant as the ecosystem changes.

SIL can therefore support:

- fixed baselines for reproducible research;
- rolling baselines for operational monitoring;
- versioned recalibration following significant network or methodology changes.

Calibration is not a one-time preprocessing step.

Structural primitives remain subject to evaluation as the network, methodology, and downstream applications evolve.

9. Machine-Readable Structural Context

SIL is designed for systems that require broad blockchain context but cannot practically reconstruct it from complete transaction histories.

Potential consumers include:

- AI agents;
- risk engines;
- automated trading systems;
- DeFi applications;
- monitoring infrastructure;
- analytics platforms.

A compact structural snapshot might resemble:

```
{
  "fragmentation": 0.27,
  "persistence_entropy": 0.61,
  "structural_drift": 0.14,
  "stability": 0.92,
  "anomaly_score": 0.03
}
```

These values are not intended to replace conventional data.

They can complement:

- price;
- liquidity;
- volume;
- volatility;
- protocol-specific metrics;
- entity intelligence;
- security signals.

The role of SIL is to provide an additional class of machine-readable context.

A more complete response can include both raw values and calibrated context:

```
{
  "network": "BNB Chain",
  "period": "2026-08-25",
  "methodology_version": "v1",
  "baseline_version": "v1-rolling",
  "structural_drift": {
    "raw_value": 0.12,
    "historical_percentile": 94,
    "context": "unusually high"
  },
  "persistence_entropy": {
    "raw_value": 0.61,
    "historical_percentile": 73,
    "context": "within expected range"
  }
}
```

This allows different users to consume structural state at the appropriate level.

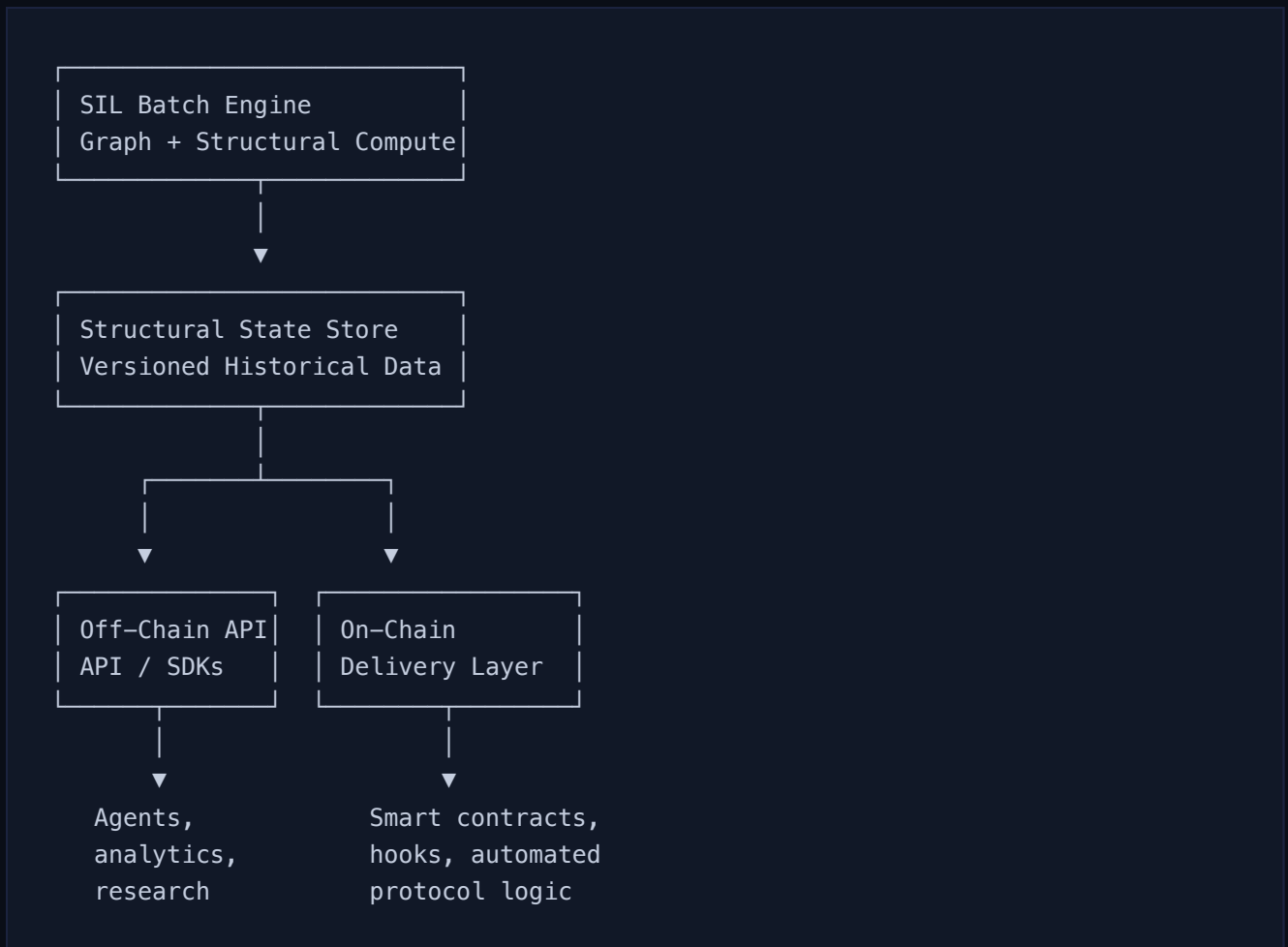
- **Researchers** can access raw primitives and methodology.
- **Analytics platforms** can use calibrated scores and historical context.
- **Automated systems** can define application-specific decision thresholds.
- **Risk systems** can combine structural deviations with other inputs.

SIL therefore aims to provide not merely structural measurements, but:

Structural measurements with documented methodology, historical context, and empirically evaluated utility.

10. Applications and Structural State Delivery

SIL delivers precomputed structural state primarily to off-chain systems, with optional mechanisms for applications that benefit from on-chain consumption.



The off-chain API is the primary delivery mechanism.

On-chain delivery is an optional extension for applications that benefit from structural primitives as direct inputs to smart-contract logic.

10.1 Structural State Publication

Each published observation includes the relevant:

- network;
- period;
- representation;
- methodology version;
- baseline version where applicable;
- primitive values.

For example:

```
{
  "network": "bnb",
  "period": "2026-08-25",
  "methodology_version": "v1",
  "baseline_version": "v1-rolling",
  "structural_drift": {
    "raw_value": 0.12,
    "historical_percentile": 94
  }
}
```

Applications should know not only the value of a primitive, but also **how that value was produced**.

10.2 Optional On-Chain Delivery

The simplest initial mechanism for on-chain consumption is a signed push architecture.

```
SIL Batch Pipeline
    ↓
Versioned Structural State
    ↓
Signed Update
    ↓
Relayer
    ↓
StructuralStateRegistry
    ↓
Protocol / Hook
```

The registry verifies that updates originate from an authorized signing entity and stores the latest valid state.

Protocols can then read the published primitives directly.

The initial implementation does not require the construction of a new decentralized oracle network.

Potential future delivery mechanisms may include:

- multiple independent data providers;
- threshold signatures;

- existing oracle infrastructure;
- replicated computation;
- optimistic verification;
- cryptographic commitments.

For applications requiring stronger guarantees regarding published historical data, SIL may also publish Merkleized or content-addressed commitments.

These mechanisms can provide verifiable provenance and integrity of published values, but do not by themselves prove the correctness of the underlying off-chain computation.

10.3 Potential On-Chain Applications

Precomputed structural primitives can potentially serve as additional inputs to smart-contract logic.

The objective is not to claim that a single primitive should directly determine protocol behavior.

Instead, structural state can be incorporated into empirically validated decision functions.

DYNAMIC PARAMETERS AND FEES

Protocols with programmable parameter logic may evaluate structural primitives alongside conventional signals.

For example, a dynamic fee model could take the general form:

$$f_t = f_{base} + \alpha \cdot volatility_t + \beta \cdot structural_drift_t$$

where the contribution of structural state is empirically calibrated.

The relevant research question is:

Does including structural state improve the behavior of the dynamic parameter relative to conventional inputs alone?

AUTOMATED RISK CONTROLS

Protocols may use structural state as an additional observable condition within predefined risk frameworks.

Potential applications include:

- changes in collateral parameters;
- reductions in exposure limits;
- temporary restrictions on selected actions;
- increased monitoring thresholds.

A structural anomaly should not automatically be interpreted as an exploit or attack.

It can instead function as an additional signal within a broader risk model.

DYNAMIC STRATEGY PARAMETERS

Automated strategies and vaults may evaluate whether structural primitives improve decisions involving:

- rebalancing frequency;
- concentration ranges;
- exposure limits;
- execution timing.

Whether these measurements improve a particular strategy remains an empirical question.

10.4 Off-Chain Applications

Off-chain systems can consume structural primitives directly without an on-chain delivery mechanism.

AI AGENTS

AI agents can query compact structural state to supplement broader blockchain context.

This can provide a compressed summary of network-level conditions without requiring an agent to independently retrieve and process the complete underlying transaction history.

ANALYTICS AND RISK PLATFORMS

Analytics platforms can combine structural primitives with:

- price;
- volume;
- liquidity;
- active addresses;
- volatility;
- protocol-specific indicators.

Structural state can therefore function as another feature family within existing analytical systems.

PROTOCOL AND ECOSYSTEM MONITORING

Protocol teams and researchers can compare structural state before and after:

- upgrades;
- deployments;
- migrations;
- liquidity events;
- market dislocations.

The resulting historical time series can support quantitative evaluation of structural change relative to network-specific baselines.

11. Open Methodology and Interoperability

SIL is intended to be developed as open infrastructure.

Public components may include:

- representation specifications;
- graph-construction methodology;
- computational pipelines;

- primitive definitions;
- calibration methodology;
- API schemas;
- SDKs;
- benchmark reports;
- validation reports;
- reference implementations.

Versioning is essential.

A structural primitive should not be an unexplained number.

Users should be able to determine:

- what representation generated it;
- which methodology version was used;
- which aggregation period it represents;
- what parameterization was applied;
- which baseline or calibration methodology provides its context.

This allows structural primitives to be treated as reproducible data products rather than opaque model outputs.

12. Limitations and Open Questions

Representation Dependence

Different representations can produce different structural results.

SIL therefore treats representation as an explicit and versioned methodological choice rather than a hidden implementation detail.

The initial production architecture focuses on graph and network representations. Alternative mathematical representations may be evaluated for specific downstream applications, but should not be assumed to belong to the core SIL data model without

separate validation.

Interpretability

A mathematically valid topological quantity does not automatically have a simple economic or operational interpretation.

Primitive definitions must therefore be evaluated for both information content and interpretability.

Computational Scaling

Persistent homology remains computationally demanding for large representations.

The initial deployment addresses this through daily aggregation, representation benchmarking, low-dimensional computation, sparsity, sampling, and empirical cost measurement.

Cross-Chain Generalization

A method that performs well on Ethereum cannot automatically be assumed to perform equally well on BNB Chain or another network.

Each deployment requires network-specific discovery, calibration, and validation.

Causality

Structural change can indicate that the network changed without determining why.

SIL is a measurement layer, not a universal causal explanation system.

Downstream Utility

A primitive may be mathematically interesting without being practically useful.

The most important long-term criterion is therefore demonstrated downstream value.

13. Initial Deployment Roadmap

Phase 1 — Discovery, Historical Validation, and Compute Benchmarking

Objectives:

- construct a historical chain-specific dataset;
- evaluate candidate structural representations;
- establish network-specific baselines;
- evaluate candidate primitives;
- validate against relevant historical events;
- benchmark runtime and memory requirements;
- evaluate sampling and coarsening strategies;
- select the initial production primitive set.

DELIVERABLES

- open-source ingestion pipeline;
- representation specification;
- historical structural dataset;
- calibration and validation report;
- **Structural Compute Benchmark Report.**

Phase 2 — Automated Daily Pipeline

Objectives:

- automate daily ingestion;
- construct daily structural representations;
- compute selected primitives;
- calculate calibrated contextual values;
- store versioned results;
- monitor computation reliability.

The initial system operates as an append-only daily dataset.

Phase 3 — Public Access and Developer Tooling

Deliverables:

- REST API;
- GraphQL interface;
- Python SDK;
- TypeScript SDK;
- documentation;
- example integrations;
- structural-state dashboard.

14. Long-Term Vision

The long-term objective is not simply to create another analytics dashboard.

It is to establish a new category of blockchain data infrastructure.

Today, developers can ask:

Which transactions occurred?
Which contracts interacted?
How much value moved?
What is the current protocol state?

A structural data layer makes additional questions programmatically accessible:

How fragmented is the interaction network?
How stable is the current structural state?
How different is today's network from its recent history?
Has a significant structural transition occurred?

The long-term vision is a versioned, reproducible, machine-readable representation of blockchain structural state.

Different networks may ultimately expose comparable families of primitives while retaining network-specific representations, calibration methods, and methodologies.

SIL can therefore provide a foundation for experimentation across:

- Layer 1 networks;
- Layer 2 networks;
- DeFi ecosystems;
- protocol interaction networks;
- transaction-level activity;
- cross-chain interaction systems.

Conclusion

Blockchain infrastructure has largely solved the problem of making blockchain activity accessible.

Transactions, events, contracts, balances, and entities can be indexed, queried, aggregated, and analyzed.

The Structural Intelligence Layer addresses a different problem:

How can the structural state of blockchain activity itself become a reusable data object?

SIL proposes a pipeline in which blockchain activity is transformed into explicit mathematical representations, analyzed using graph and topological methods, and published as versioned structural primitives.

The approach is supported by empirical experiments across structural anomaly detection, predictive validation, and MEV classification. These experiments do not validate every future application of SIL, but they provide evidence that structural representations can contain measurable information across different blockchain tasks and representations.

The production design deliberately separates expensive structural computation from user-facing access. The initial system operates through daily offline computation and precomputed storage, while compute cost is treated as an explicit benchmarking objective rather than an unsupported assumption.

A central part of the methodology is that primitives are not assumed to be universally meaningful.

Each new deployment begins with:

Discover → Evaluate → Calibrate → Validate → Publish

The proposal rests on four principles:

1. **Blockchain networks possess structural properties that are not fully represented by conventional aggregate metrics.**
2. **Graph and topological methods can produce measurable descriptors of those properties.**
3. **Structural primitives should be selected, calibrated, and validated empirically rather than assumed to be universally meaningful.**
4. **Where these descriptors demonstrate utility, they should be transformed from bespoke analytical pipelines into reusable public infrastructure.**

SIL does not claim that topology replaces graph analytics, indexing infrastructure, or conventional blockchain metrics.

It proposes a complementary layer.

Indexers make blockchain activity queryable.

SIL proposes to make blockchain structure queryable.

Appendix A: Mathematical Terms and Primitive Glossary

This appendix provides concise definitions of the principal mathematical concepts used in the Structural Intelligence Layer. Exact implementations, representations, and parameterizations are versioned and may vary between deployments.

Structural primitives

- **Betti Number (β_0)** — Number of connected components in a topological representation. Used as a measure of structural fragmentation.
- **First Betti Number (β_1)** — Number of independent topological loops. Used to characterize cyclical or interconnected structure.
- **Persistence** — Measures how long a topological feature remains present across a range of scales. Longer persistence generally indicates a more structurally significant feature.
- **Persistence Entropy** — Measures how persistence is distributed across topological features. It provides a compact measure of the diversity of structural scales present.
- **Total Persistence** — The sum of persistence lifetimes across topological features. Used as a measure of the overall amount of persistent structure.
- **Structural Drift** — A distance between two persistence diagrams, computed using the Wasserstein distance. It measures how much the observed topological structure has changed between periods.
- **Topology Velocity** — The rate of structural change over time, conceptually expressed as structural drift divided by elapsed time.

Core concepts

- **Filtration** — A sequence of increasingly permissive mathematical representations through which structural features appear and disappear across scale.
- **Persistence Diagram** — A representation of topological features by their birth and death scales. It provides the underlying object from which persistence-based primitives are derived.
- **Persistent Homology** — A method from Topological Data Analysis that tracks the appearance and disappearance of topological features across a filtration.

- **Wasserstein Distance** — A metric that quantifies the difference between two persistence diagrams by optimally matching their topological features. SIL uses Wasserstein distance to quantify structural drift.

These quantities do not have universal economic interpretations. Their meaning depends on the underlying representation, filtration, aggregation period, and calibration methodology used by a particular SIL deployment.